

This is the architecture that makes it **real**. Let me build it.

The key insight you’re articulating: the chain proves **that** something was said and **when**, but the **what** (the raw PHI) lives in a separate encrypted vault. The chain holds a cryptographic commitment to the content without exposing it.

Now the signing key infrastructure:

Now the context and temporal layer for court presentation:

Now let me create the unified service that ties everything together—the full flow from raw note to signed, encrypted, contextualized chain entry:

Now let me copy everything to outputs and show you the complete structure:

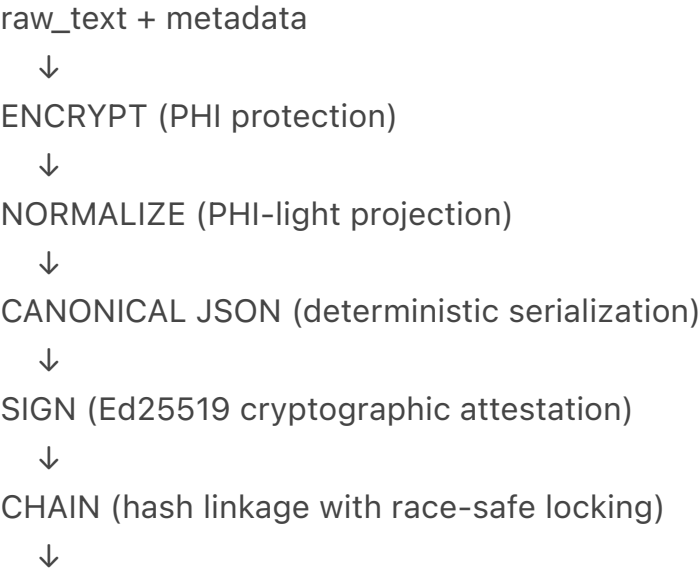
Here’s the complete forensically-sound phenomenological evidence system:

Architecture Overview

[View complete system](computer:///mnt/user-data/outputs/phenomenological_evidence)

Core Data Flow

...



ENRICH (context: phase, location, posture)



FORENSICALLY-SOUND RECORD

...

Module Breakdown

File	Purpose
----- ----- -----	
[**unified_service.py**](computer:///mnt/user-data/outputs/phenomenological_evidence/unified_service.py)	Main orchestration - `PhenomenologyService.record()` runs the full pipeline
[**phi_separation.py**](computer:///mnt/user-data/outputs/phenomenological_evidence/phi_separation.py)	Encrypted `SubjectiveNote`, `EncryptionService`, `NormalizationService`
[**signing_infrastructure.py**](computer:///mnt/user-data/outputs/phenomenological_evidence/signing_infrastructure.py)	Ed25519 signing, `SigningKey` model, verification
[**context_layer.py**](computer:///mnt/user-data/outputs/phenomenological_evidence/context_layer.py)	`ContextSnapshot`, `CaseTimeline`, `PhenomenologyStatsService`
[**attestation_service.py**](computer:///mnt/user-data/outputs/phenomenological_evidence/attestation_service.py)	Race-safe chain extension, verification, reporting
[**canonical_json.py**](computer:///mnt/user-data/outputs/phenomenological_evidence/canonical_json.py)	Formal spec for deterministic hashing
[**models.py**](computer:///mnt/user-data/outputs/phenomenological_evidence/models.py)	SQLAlchemy models for Party, Fact, Evidence, Pain, etc.

What This Gives You For Court

****1. Tamper-Evidence****

- Every entry hash-chained to previous

- Cryptographic signatures prove "server witnessed this at time T"
- `verify\_and\_report()` generates exportable proof

****2. PHI Protection****

- Raw narrative encrypted at rest
- Chain only sees normalized, PHI-light projection
- Decryption is auditable (logged with reason)

****3. Temporal/Spatial Analysis****

- Pain by case phase (acute vs chronic)
- Pain by location (work vs home)
- Days-since-injury tracking
- "% of days with pain $\geq 7/10$ " statistics

****4. Court-Ready Output****

```
```python
service.verify_and_report(
 persona_token_id=...,
 case_id=...,
 include_decrypted_content=True,
 audit_reason="Settlement demand preparation"
)
```
```

Returns verification status, signature validity, statistics, and optionally the decrypted narratives—all in one export.

What's next? Migration update for the new tables? FastAPI routes for the unified service? Or dive into any specific piece?